Needy Bot

Autonomous Delivery Rover

By

Daniyal Ahmed, Danish Kumar, Ethan Collier

Table of Contents

1. Introduction 1
1.1 Literature Review 1
1.2 Goal of Project 1
2. Methodology 1
2.1 Overview 1
2.2 Application 3
2.2.1 Hardware 3
2.2.1.1 Structure 3
2.2.1.2 Circuit Diagram 5
2.2.1.3 Table of Components 6
2.2.2 Software 7
2.2.2.1 Overview 7
2.2.2.2 Sensing 8
2.2.2.3 Motion and Navigation 10
3. Results 13
3.1 Testing 13
3.2 Conclusion 14
3.3 Future Work 15
References: 16
Appendices 17
Appendix A – Arduino Firmware Code 17
Appendix B – Python Dashboard Code for Raspberry Pi 20
Appendix C – Use of Generative AI 56

1. Introduction

1.1 Literature Review

With the introduction of rule-based systems, autonomous mobile robots have evolved into intelligent systems. Brooks (1986) introduced behavior-based robotics, providing an example of layered control in reaction to environmental events [1]. As the computational power increased, so did the use of probabilistic approaches. Thrun, Burgard, and Fox (2005) introduced probabilistic robotics, including uncertainty in robot localization and mapping (SLAM) [2]. Earlier, Fox, Burgard, and Thrun (1999) introduced probabilistic localization using probabilistic filters, such as Markov localization [3]. Siegwart, Nourbakhsh, and Scaramuzza (2011) have provided a framework for exploring the mechanics, motion planning, and multi-sensor systems required for autonomous mobile robots [4].

Recent work frequently emphasizes the use of fusion of several sensing systems to effectively navigate indoors. The integration of vision, supported by libraries like OpenCV [5], enables object detection and environment perception. But these systems are vulnerable to environmental factors such as lighting and shadows. Combining sensing modes such as LiDAR [7], ultrasonic, inertial, and encoder data can be used to find obstacles and make precise movements in an indoor space [8], [9]. Distributed control architectures are used in robot design to provide increased computation efficiency and react quickly to time-critical tasks [6]. This relieves the top-level computations while providing quick responses to functions on the lower level, leading to increased stability and responsiveness [10].

1.2 Goal of Project

The goal of this project was to develop an affordable autonomous delivery rover capable of safe indoor navigation using a multi sensor approach, with a web based dashboard for remote monitoring and control.

2. Methodology

2.1 Overview

The Needy Bot system was built on a Lynxmotion A4WD1 four-wheel-drive chassis using a dual-controller architecture. A Raspberry Pi 3B served as the high-level processing unit responsible for sensor data acquisition, navigation decision-making, computer vision, and hosting a web-based control dashboard. An Arduino Uno served as the low-level controller responsible for real-time motor control through a Sabertooth 12A dual motor driver. Three sensors provided environmental awareness: a Slamtec RPLIDAR A1 for 360-degree scanning, an HC-SR04 ultrasonic sensor for close-range emergency stopping, and a Universal Serial Bus (USB) camera for visual obstacle detection and line following. A custom three-dimensionally printed delivery bin with an external RPLIDAR pole mount provided cargo capacity. The system supported three operating modes: manual teleoperation, autonomous LIDAR-based navigation, and camera-based line following. The system architecture is shown in Figure 1.

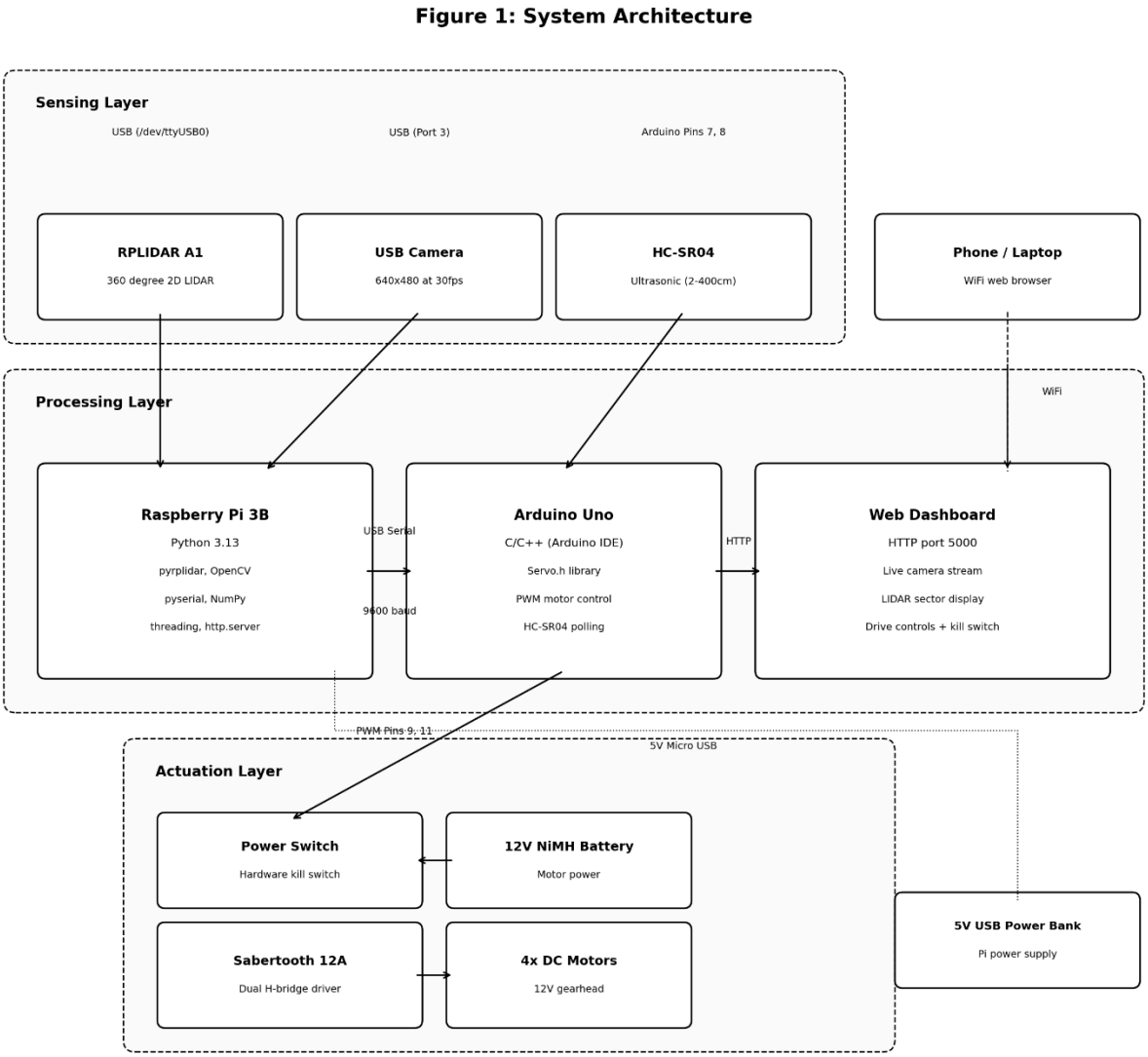

Figure 1: Three-layer system architecture

2.2 Application

2.2.1 Hardware

2.2.1.1 Structure

The mechanical foundation was a Lynxmotion A4WD1 aluminum chassis measuring 203 mm wide by 248 mm long with a height of 102 mm. The chassis was constructed from heavy-duty anodized aluminum structural brackets and laser-cut Lexan panels, providing a payload capacity of approximately 2.3 kg. A custom delivery bin was designed using AutoCAD and manufactured in Polyethylene Terephthalate Glycol (PETG) using a Bambu Lab three-dimensional printer. The bin measured 170 mm by 170 mm by 110 mm with 3 mm wall thickness and was mounted on the top deck of the chassis.

An external L-bracket pole mount was designed and three-dimensionally printed to hold the RPLIDAR A1 above the bin, ensuring an unobstructed 360-degree scan plane. The pole was 160 mm tall with a 22 mm outer diameter hollow tube allowing the USB cable to be routed internally.

Motors

Four 12V direct current (DC) gearhead motors were arranged in a tank-drive configuration, with the left pair connected to motor channel 1 and the right pair connected to motor channel 2 on the Sabertooth 12A dual motor driver. The Sabertooth received Pulse Width Modulation (PWM) servo signals from the Arduino Uno on pins 9 and 11, where a servo value of 90 represented stop, values above 90 produced forward motion, and values below 90 produced reverse motion. The DIP switches on the Sabertooth were configured to 011010, enabling exponential response and Nickel Metal Hydride (NiMH) battery mode.

Sensors

The Slamtec RPLIDAR A1 provided 360-degree two-dimensional laser scanning at approximately 5.5 Hz with roughly 277 measurement points per revolution and a maximum range of 12 meters. The RPLIDAR was connected to the Raspberry Pi via a USB adapter board. The pyrplidar Python library was used to interface with the sensor, specifically the start_scan_express method which was determined to be the most reliable scan mode for the installed firmware version.

An HC-SR04 ultrasonic distance sensor was mounted at the front of the chassis on a small breadboard, providing close-range obstacle detection from 2 cm to 400 cm. The sensor was connected to the Arduino Uno with the trigger pin on digital pin 7 and the echo pin on digital pin 8. A USB camera providing 640 by 480 resolution at 30 frames per second was mounted on the front face of the delivery bin.

Control and Power

The dual-controller architecture used USB serial communication at 9600 baud between the Raspberry Pi and Arduino Uno. A physical power switch was installed on the positive battery lead

between the 12V NiMH battery and the Sabertooth B+ terminal as a hardware kill switch. A separate 5V USB power bank supplied the Raspberry Pi through its micro USB power input. The complete wiring diagram showing all connections is presented in Figure 3.

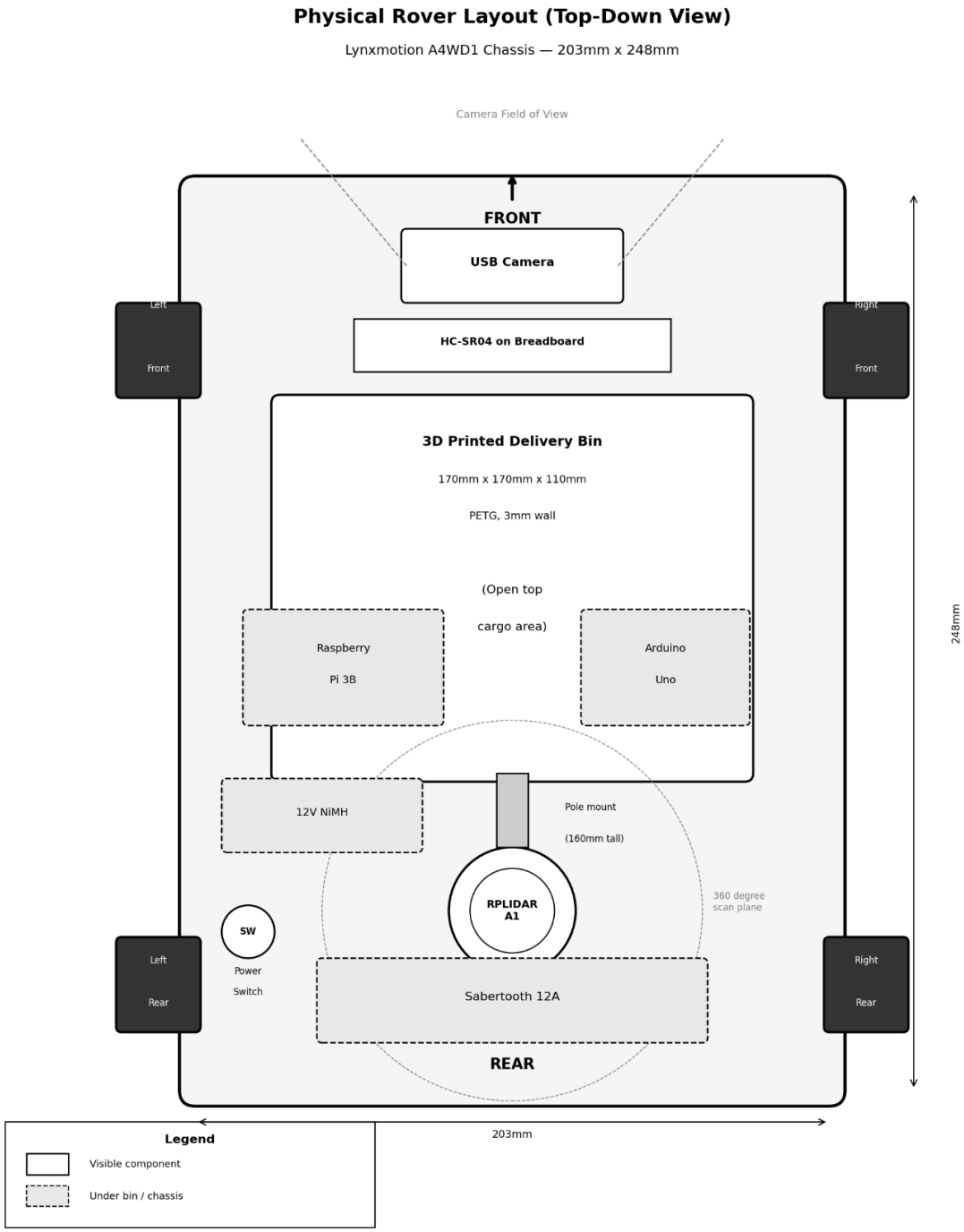

Figure 2: Physical Rover Layout View

2.2.1.2 Circuit Diagram

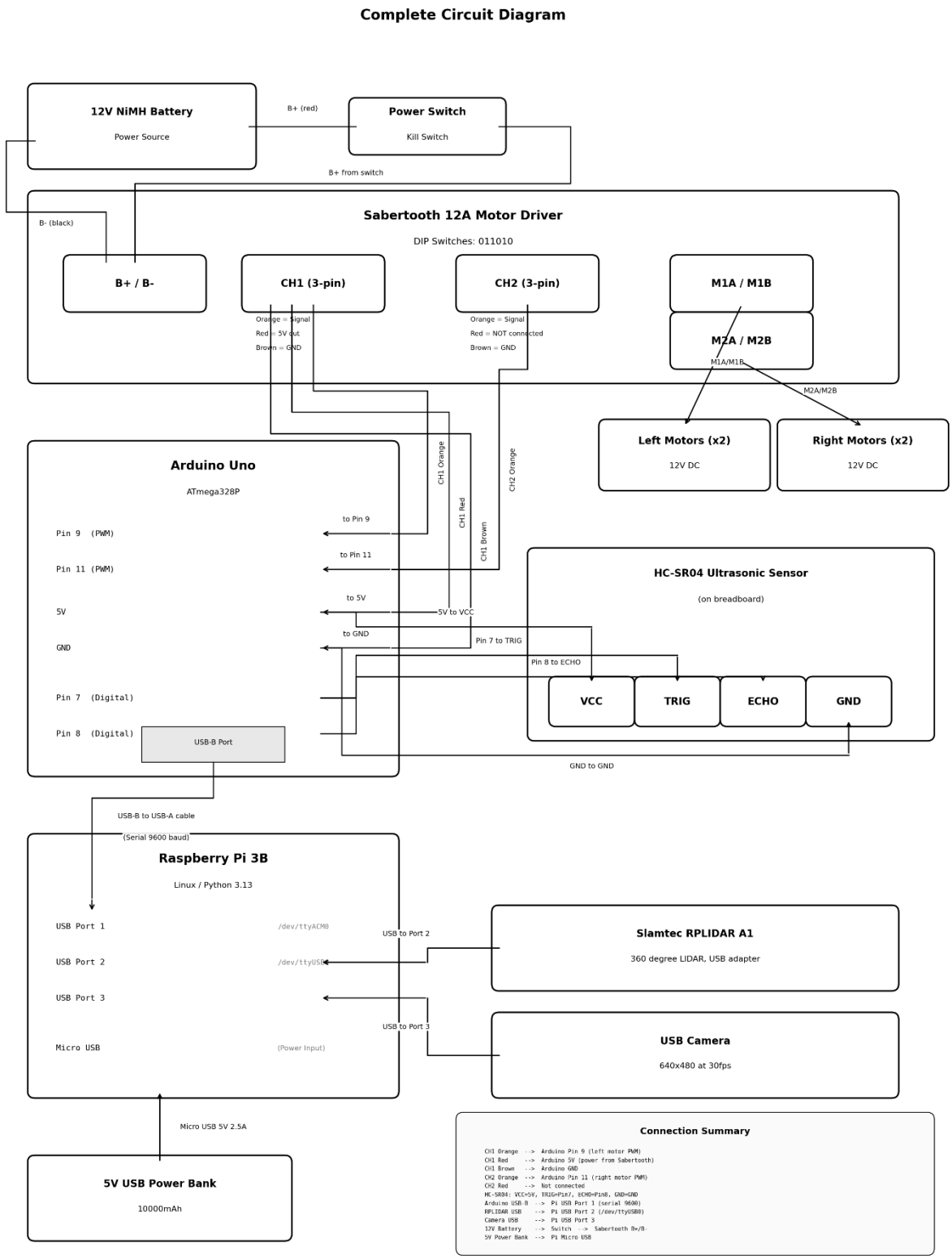

Figure 3: Complete Circuit Diagram for Wired Network

2.2.1.3 Table of Components

Complete List of the hardware components is presented in Table 1.

Table 1: Bill of Materials

Component	Model	Qty	Interface	Function
Chassis	Lynxmotion A4WD1	1	N/A	Mechanical Platform
Motor Driver	Sabertooth 12A	1	PWM	Dual H-bridge Motor Control – helps to control left side and right-side motors to work parallelly
DC motors	12V gearhead	4	Sabertooth	Wheel Drive
Single Board Computer	Raspberry pi 3B	1	USB, Wi-Fi, Micro USB	Autonomous navigation, vision, web server
Microcontroller	Arduino Uno	1	USB serial	Realtime motor controller
2D LiDAR	RPLiDAR A1	1	USB	360 degree range scanning
Camera	USB camera	1	USB	Visual perception, streaming
Distance Sensor	HC-SR04	1	GPIO	Close range emergency stop
Battery	12V NiMH	1	N/A	Motor and Arduino Power supply
Power Bank	5V USB – 8000 mAH	1	Micro USB	Raspberry Pi power supply
Delivery Bin	3D printed PETG	1	N/A	Cargo enclosure on top of the chassis
Tower	3D printed PETG	1	N/A	Place to mount Lidar attached to bin get the clear view for 360 area

2.2.2 Software

2.2.2.1 Overview

The software architecture followed a client-server model implemented across two processors. The Arduino Uno ran firmware written in C and C++ using the Arduino Integrated Development Environment (IDE), utilizing the built-in Servo.h library for PWM motor control. The Raspberry Pi 3B ran Python 3.13 scripts for all high-level functionalities. The key Python libraries used were pyrplidar version 0.1.2 for RPLIDAR A1 communication, OpenCV for camera image processing, pyserial for USB serial communication with the Arduino, NumPy for numerical array operations, and the built-in http.server module for the web dashboard. Threading was managed using the Python threading module with Lock objects for thread-safe serial port access. The operating mode state diagram is shown in Figure 4.

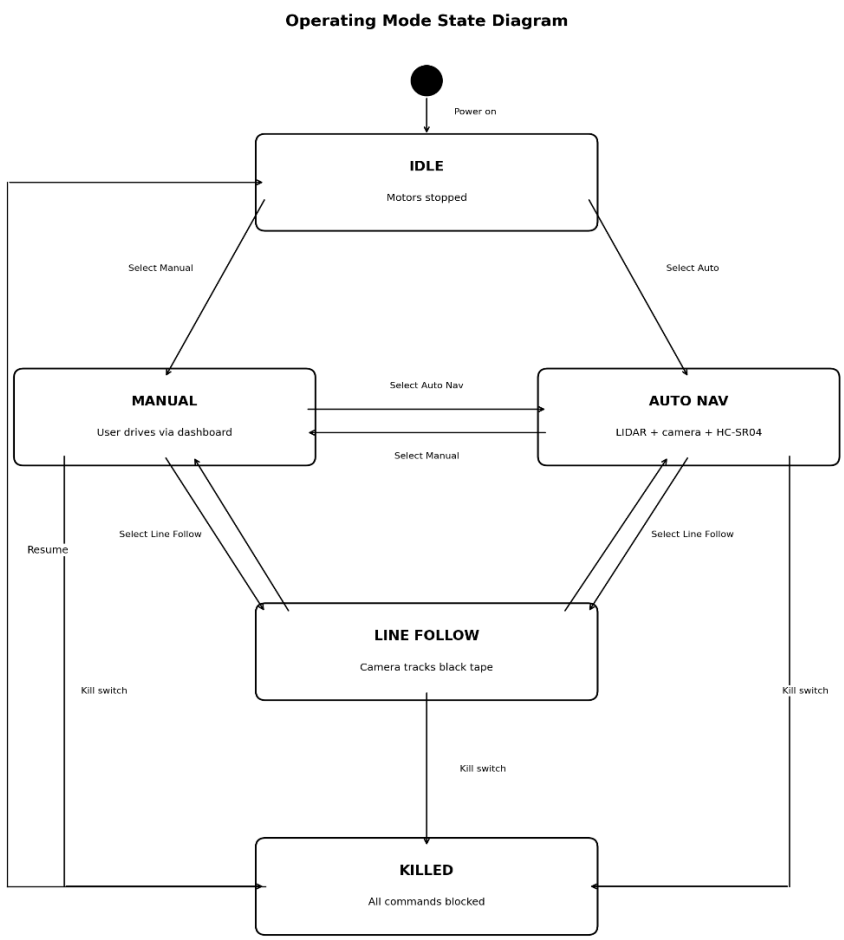

Figure 4: Operating Mode State Diagram

2.2.2.2 Sensing

The RPLIDAR A1 sensor data was acquired in a dedicated background thread running continuously during operation. The 360-degree scan was divided into six directional sectors: front (340 to 20 degrees), front-left (20 to 60 degrees), front-right (300 to 340 degrees), left (60 to 120 degrees), right (240 to 300 degrees), and back (150 to 210 degrees). The minimum valid distance in each sector was computed, with readings below 150 mm filtered out as rover body reflections.

The HC-SR04 ultrasonic sensor was polled in a separate background thread every 300 milliseconds. A threading Lock was implemented to synchronize all serial port access between the distance polling thread and the motor command function, as concurrent serial writes caused data corruption and unreliable readings.

The USB camera was processed in the navigation thread at approximately 30 frames per second. OpenCV was used to convert frames to grayscale, apply Gaussian blur for noise reduction, and perform Canny edge detection on the lower portion of the frame to estimate proximity to obstacles. The Unified Modeling Language (UML) class diagram showing all software components is presented in Figure 5.

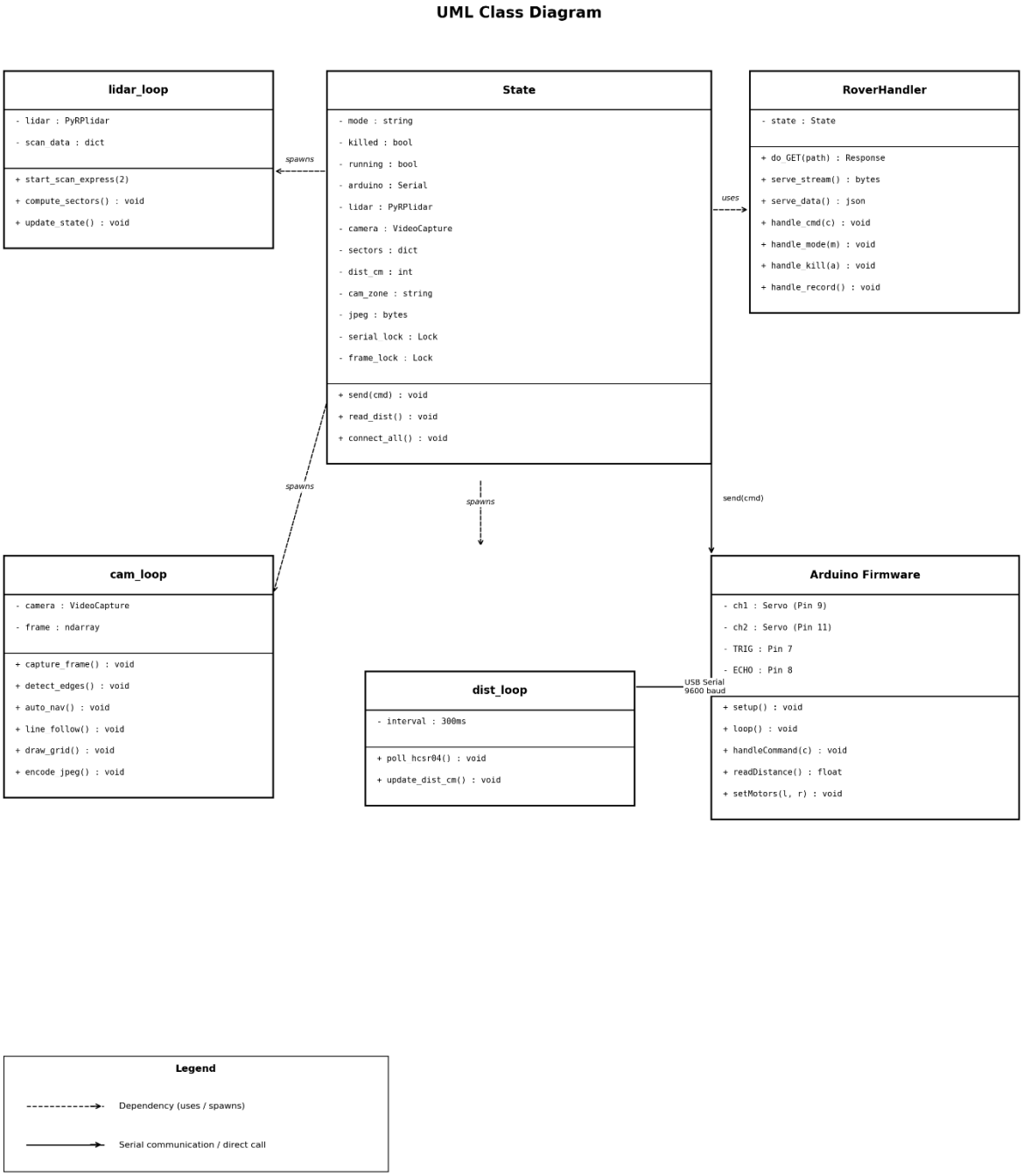

Figure 5: UML class diagram

2.2.2.3 Motion and Navigation

The Arduino firmware implemented a command interpreter that received single-character American Standard Code for Information Interchange (ASCII) commands over USB serial at 9600 baud. The command set included forward (F, servo value 105), reverse (b, servo value 78), pivot right (R), pivot left (L), gentle right curve (r, servo values 92 and 100), gentle left curve (l, servo values 100 and 92), stop (S, servo value 90), distance query (D), and ping (P). Each command was acknowledged with a response.

The autonomous navigation algorithm implemented a reactive obstacle avoidance strategy with a priority-based decision hierarchy. The highest priority was assigned to emergency stop conditions triggered by the HC-SR04 reading below 18 cm or the camera danger zone exceeding 30 percent edge fill. The second priority was LIDAR-based front sector blockage below 800 mm, which triggered a timed stop-reverse-turn sequence. The navigation decision flowchart is shown in Figure 6.

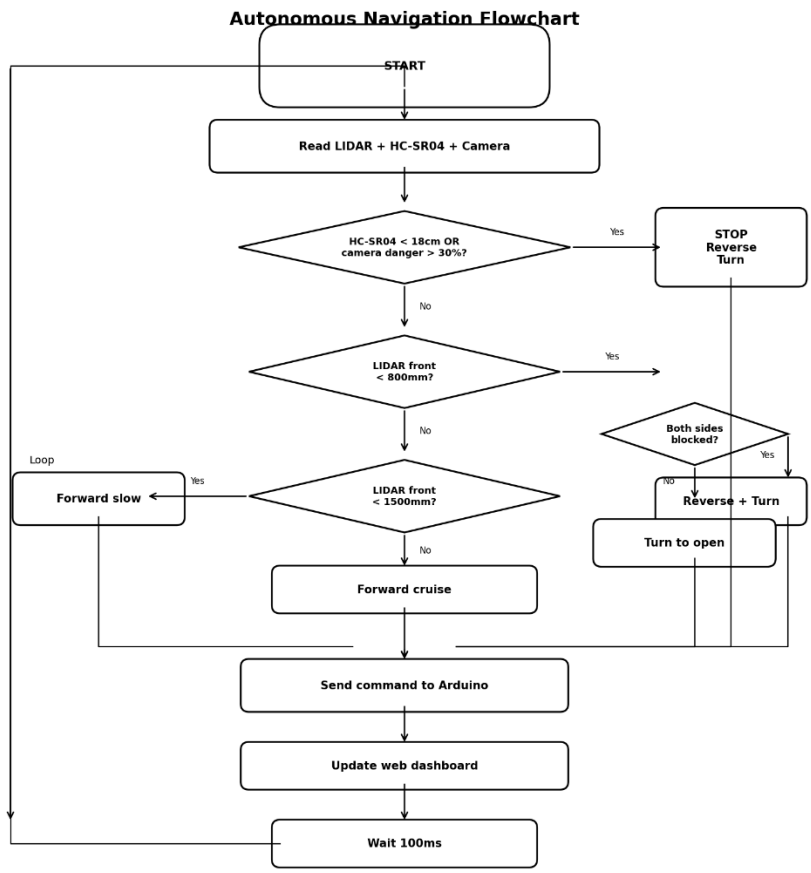

Figure 6: Autonomous Navigation Flowchart

The line-following algorithm used the bottom 20 percent of the camera frame as a region of interest. The grayscale image was thresholded using OpenCV with binary inverse thresholding to detect dark electrical tape on a light-colored floor. Morphological opening and closing operations were applied to remove noise. The largest contour was identified as the tape line, and its centroid was calculated using image moments. The horizontal error between the centroid and the frame center determined the steering command.

A web-based dashboard was implemented as a Hypertext Transfer Protocol (HTTP) server on port 5000, accessible from any device on the same WiFi network. The dashboard provided a live camera stream, real-time LIDAR sector distance display, manual drive controls, mode selection, video recording, and a software emergency stop button. The software component dependencies are shown in Figure 7 and the message sequence for one navigation cycle is shown in Figure 8.

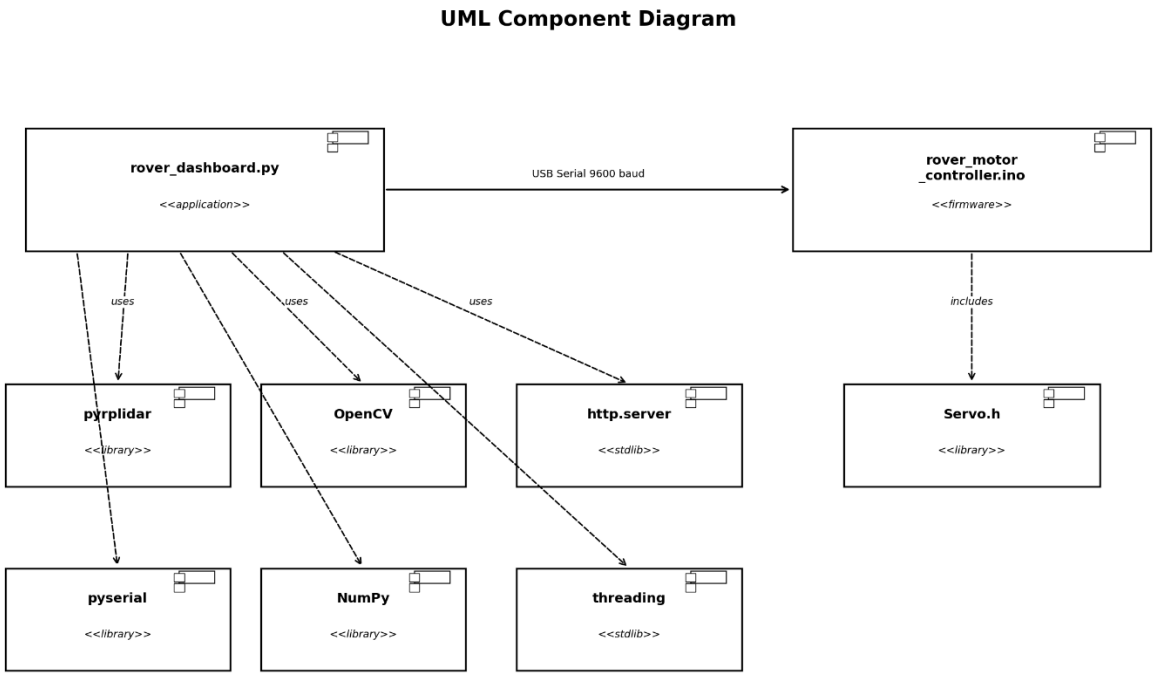

Figure 7: UML Component Diagram

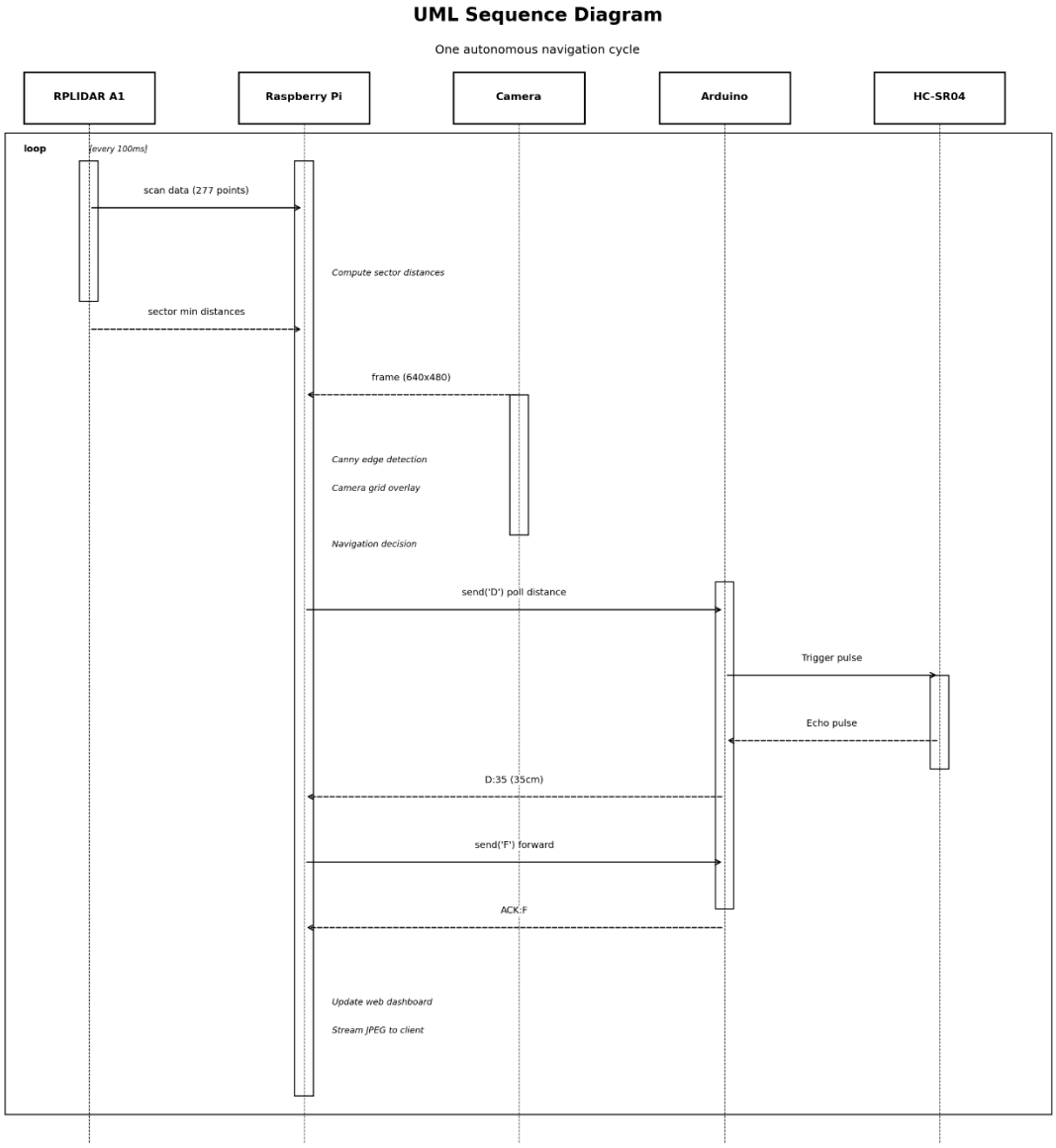

Figure 8: UML Sequence Diagram

3. Results

3.1 Testing

Component-level and integration testing was performed to verify each subsystem. The results are summarized in Table 2.

Table 2: Test Results Summary

Count	Description	Result	Status
1	Arduino Serial Ping	Responded OK with 50 ms	Pass
2	HC-SR04 Accuracy	Read 35 cm, actual 34 cm	Pass
3	Motor Commands	All wheels responded correctly	Pass
4	LiDAR scan data	277 points per revolution, 108 to 6470 mm	Pass
5	Camera Capture	640x480 at 30fps confirmed	Pass
6	Pi-Arduino Serial Link	Commands received and acknowledged	Pass
7	Web Dashboard Access	Accessible from phone and laptop	Pass
8	Manual Mode Control	All directions responded correctly	Pass
9	Camera Stream Latency	Approximately 200 ms delay	Pass
10	Lidar sector display	Updated at 5 Hz on dashboard	Pass
11	Auto Obstacle avoidance	Avoided large obstacles, narrow once missed	Partial
12	Emergency Stop	Triggered at around 13 cm to 18 cm range	Pass
13	Camera Obstacle Grid	Grid displayed correctly	Pass
14	Line Following	Detects line, sharp turn needs tuning	Partial
15	Software Kill Switch	Motor stops immediately	Pass
16	Hardware Kill Switch	Battery powers cut immediately	Pass
17	Video Recording	Avi file saved in Pi storage	Pass

Tests 1 through 10 confirmed correct operation of all individual components and communication links. The RPLIDAR A1 produced consistent scan data with approximately 277 points per revolution, detecting objects from 108 mm to 6740 mm in an indoor environment. The web dashboard was successfully accessed from both a laptop and a mobile phone over WiFi, with the camera stream displaying at approximately 200 milliseconds of latency.

Test 11, autonomous obstacle avoidance, achieved partial success. The rover successfully detected and avoided large obstacles such as walls and furniture when LIDAR data was properly received. An intermittent issue with the pyrplidar library required switching to the start_scan_express method. Narrow obstacles such as chair legs occasionally fell between LIDAR scan points.

Test 14, line following, achieved partial success. The system correctly detected black electrical tape on a light floor and tracked it along straight paths. Sharp turns and intersections sometimes caused the line to be lost, and other dark objects were occasionally misidentified as the line. Below are the some visual proofs attached of the testing of the rover's movement.

Figure 9: Rover in Automation Mode

Figure 10: Rover Following Line follow mode

Figure 11: Rover made emergency stop

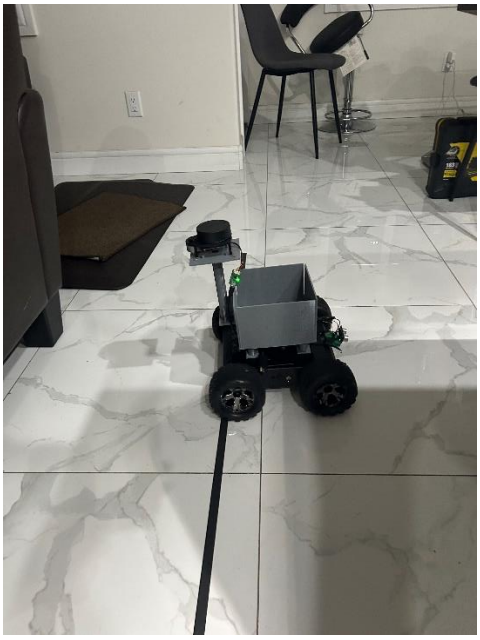

Figure 12: Rover is trying to calibrate and detect best direction to follow

3.2 Conclusion

The project successfully demonstrated a functional autonomous delivery rover constructed from commercially available, low-cost components. The dual-controller architecture effectively

separated real-time motor control from computationally intensive sensor processing.The web-based dashboard provided a practical interface for monitoring and controlling the rover remotely.

The multi-sensor approach combining RPLIDAR, ultrasonic sensing, and camera-based detection provided layered environmental awareness. Manual mode operated reliably with responsive controls. Autonomous navigation avoided most obstacles but required further tuning for complex environments. Line following worked on straight paths but needed improvement for sharp curves. The three-dimensionally printed delivery bin with external RPLIDAR pole mount preserved full cargo space while maintaining sensor coverage. The project goals were largely met as demonstrated by the test results in Table 2.

3.3 Future Work

Several areas for future development were identified. Implementation of a full SLAM algorithm such as Hector SLAM would enable map-based path planning and waypoint navigation. Integration of deep learning object detection models would improve obstacle classification. The line-following algorithm could be improved through adaptive thresholding and Proportional-Integral-Derivative (PID) control for smoother curved path tracking. Addition of wheel encoders would enable odometry-based position tracking. Hardware improvements could include upgrading to a Raspberry Pi 5 for increased processing power and adding an Inertial Measurement Unit (IMU) for improved orientation estimation.

References

[1] R. A. Brooks, "A robust layered control system for a mobile robot," IEEE J. Robot. Autom., vol. 2, no. 1, pp. 14-23, 1986.

[2] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.

[3] D. Fox, W. Burgard, and S. Thrun, "Markov localization for mobile robots in dynamic environments," J. Artif. Intell. Res., vol. 11, pp. 391-427, 1999.

[4] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA: MIT Press, 2011.

[5] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library. Sebastopol, CA: O'Reilly Media, 2008.

[6] R. Boudali et al., "Mobile robot platform with Arduino Uno and Raspberry Pi for autonomous navigation," Procedia Manuf., vol. 32, pp. 874-880, 2019.

[7] P. Ogunlana, O. Adeshina, and I. Adekunle, "The impact of LiDAR configuration on goal-based navigation within a deep reinforcement learning framework," Sensors, vol. 23, no. 24, Dec. 2023.

[8] D. P. Kurniawan et al., "Implementation of LiDAR sensor for mobile robot delivery based on robot operating system," J. Phys. Conf. Ser., 2024.

[9] S. S. Sreelekshmi et al., "LiDAR based autonomously navigating robot," in Proc. IEEE Conf., 2024.

[10] Slamtec, "RPLIDAR A1 360 Degree Laser Scanner Development Kit Datasheet," Shanghai Slamtec Co., Ltd., 2024.

Appendices

Appendix A – Arduino Firmware Code

```
* rover_motor_controller.ino
 * Autonomous Delivery Rover — Arduino Uno Firmware
 *
 * Hardware:
 *   - Sabertooth 12A motor driver on pins 9 (CH1/left) and 11 (CH2/right)
 *     via Servo library (RC mode). Servo 90 = stop, >90 = fwd, <90 = rev.
 *   - HC-SR04 ultrasonic sensor: TRIG=7, ECHO=8
 *   - Serial at 9600 baud with Raspberry Pi
 *
 * Commands (single character):
 *   F  — Forward        → ACK:F
 *   b  — Reverse        → ACK:b
 *   R  — Pivot right    → ACK:R
 *   L  — Pivot left     → ACK:L
 *   r  — Gentle right   → ACK:r
 *   l  — Gentle left    → ACK:l
 *   S  — Stop           → ACK:S
 *   D  — Distance reading → D:<cm>  (or D:-1 on timeout)
 *   P  — Ping           → OK
 *
 * Speed values are intentionally conservative — closer to neutral (90)
 * means slower and safer indoors. Increase toward 105/78 if more speed needed.
 */

#include <Servo.h>

// — Pin assignments ————————————————————————————————————————
const int PIN_LEFT  =  9;  // CH1 — left  motors (Sabertooth S1)
const int PIN_RIGHT  = 11;  // CH2 — right motors (Sabertooth S2)
const int PIN_TRIG  =  7;  // HC-SR04 trigger
const int PIN_ECHO  =  8;  // HC-SR04 echo

// — Speed values (Servo degrees) ————————————————————————————
// Neutral = 90. Increase → faster forward. Decrease → faster reverse.
// Keep these close to 90 for safe indoor use.
const int SPD_STOP       = 90;

const int SPD_FORWARD     = 100;  // straight forward  (slowed from 105)
const int SPD_REVERSE     = 80;  // straight reverse  (slowed from 78)
```

```
// Pivot MUST be at least 10 units from neutral (90) to clear Sabertooth deadband.
// Values like 96/84 are inside the deadband and cause NO movement.
const int SPD_PIVOT_FWD    = 100;  // pivot: forward side
const int SPD_PIVOT_REV    = 80;   // pivot: reverse side

const int SPD_GENTLE_OUTER = 97;   // gentle curve: faster wheel
const int SPD_GENTLE_INNER = 93;   // gentle curve: slower wheel

// ── HC-SR04 timeout ──────────────────────────────────────────────────────
const unsigned long ECHO_TIMEOUT_US = 30000UL;   // 30 ms ≈ 5 m max

// ── Objects ──────────────────────────────────────────────────────────────
Servo leftMotors;
Servo rightMotors;

// ── Helpers ──────────────────────────────────────────────────────────────

inline void setMotors(int left, int right) {
  leftMotors.write(left);
  rightMotors.write(right);
}

long readDistanceCm() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  long duration = pulseIn(PIN_ECHO, HIGH, ECHO_TIMEOUT_US);
  if (duration == 0) return -1L;
  return (duration * 34L) / 2000L;
}

// ── Setup ────────────────────────────────────────────────────────────────
void setup() {
  Serial.begin(9600);

  leftMotors.attach(PIN_LEFT);
  rightMotors.attach(PIN_RIGHT);
  setMotors(SPD_STOP, SPD_STOP);   // safety: start stopped

  pinMode(PIN_TRIG, OUTPUT);
```

```
  pinMode(PIN_ECHO, INPUT);
  digitalWrite(PIN_TRIG, LOW);

  delay(50);
}

// — Main loop ————————————————————————————————————————————————
void loop() {
  if (!Serial.available()) return;

  char cmd = Serial.read();

  switch (cmd) {

    // — Forward ————————————————————————————————————————
    case 'F':
      setMotors(SPD_FORWARD, SPD_FORWARD);
      Serial.println("ACK:F");
      break;

    // — Reverse ————————————————————————————————————————
    case 'b':
      setMotors(SPD_REVERSE, SPD_REVERSE);
      Serial.println("ACK:b");
      break;

    // — Pivot right: left fwd, right rev ————————————————————
    case 'R':
      setMotors(SPD_PIVOT_FWD, SPD_PIVOT_REV);
      Serial.println("ACK:R");
      break;

    // — Pivot left: left rev, right fwd ————————————————————
    case 'L':
      setMotors(SPD_PIVOT_REV, SPD_PIVOT_FWD);
      Serial.println("ACK:L");
      break;

    // — Gentle right: both fwd, right side slightly slower ————————
    case 'r':
      setMotors(SPD_GENTLE_OUTER, SPD_GENTLE_INNER);
      Serial.println("ACK:r");
      break;
```

```
  // — Gentle left: both fwd, left side slightly slower ————————
  case 'l':
    setMotors(SPD_GENTLE_INNER, SPD_GENTLE_OUTER);
    Serial.println("ACK:l");
    break;

  // — Stop ————————————————————————————————————————————
  case 'S':
    setMotors(SPD_STOP, SPD_STOP);
    Serial.println("ACK:S");
    break;

  // — Distance (HC-SR04) ————————————————————————————————
  case 'D': {
    long cm = readDistanceCm();
    Serial.print("D:");
    Serial.println(cm);
    break;
  }

  // — Ping ————————————————————————————————————————————
  case 'P':
    Serial.println("OK");
    break;

  default:
    break;
 }
}
```

Appendix B – Python Dashboard Code for Raspberry Pi

```
#!/usr/bin/env python3
"""
rover_dashboard.py — Autonomous Delivery Rover Dashboard
============================================================
Hardware:  Raspberry Pi 3B + Arduino Uno (via USB serial)
           + Slamtec RPLIDAR A1 (via USB)
           + USB Camera (640×480)
           + HC-SR04 via Arduino

Run:  python3 rover_dashboard.py
Open: http://<pi-ip>:5000
```

```
Threading model
──────────────────────
  Main        → ThreadedHTTPServer (one thread per connection)
  lidar_t     → RPLIDAR scan loop (start_scan_express only)
  camera_t    → capture + display + line-follow commands
  distance_t  → HC-SR04 poll every 300 ms
  nav_t       → auto-nav decision loop every 200 ms

All serial access is protected by serial_lock.
Frame buffer is protected by frame_lock.
Shared state writes are protected by state_lock.
"""

import sys
import os
import io
import json
import math
import time
import signal
import socket
import datetime
import threading
from http.server import HTTPServer, BaseHTTPRequestHandler
from socketserver import ThreadingMixIn

# ── Optional library imports ──────────────────────────────────────────────────
try:
    import serial
    HAS_SERIAL = True
except ImportError:
    HAS_SERIAL = False
    print("[WARN] pyserial not installed — serial features disabled")

try:
    import cv2
    import numpy as np
    HAS_CV = True
except ImportError:
    HAS_CV = False
    print("[WARN] OpenCV (cv2) not installed — camera features disabled")

try:
    from pyrplidar import PyRPlidar
```

```
    HAS_LIDAR = True
except ImportError:
    HAS_LIDAR = False
    print("[WARN] pyrplidar not installed — LIDAR disabled")

#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════
#  CONSTANTS
#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════

HTTP_PORT    = 5000
SERIAL_BAUD  = 9600

# Arduino USB serial candidates (ACM = CH340/ATmega16U2, not USB0/1 used by LIDAR)
ARDUINO_PORTS = ['/dev/ttyACM0', '/dev/ttyACM1', '/dev/ttyACM2',
                 '/dev/ttyUSB2', '/dev/ttyUSB3']

# LIDAR USB candidates (USB0 primary, USB1 fallback)
LIDAR_PORTS = ['/dev/ttyUSB1', '/dev/ttyUSB0']   # USB1 is where LIDAR appears on this Pi

CAMERA_IDX = 0   # try 0 first, then 1

# Modes
MODE_MANUAL = 'manual'
MODE_AUTO   = 'auto_nav'
MODE_LINE   = 'line_follow'

# ── LIDAR sector definitions
──────────────────────────────────────────────────────────────────────────────
# Angles in degrees. start > end means the sector crosses 0°.

#genAI Helped finding these numbers
SECTORS = {
    'front':       (340, 20),
    'front_left':  (20,  60),
    'front_right': (300, 340),
    'left':        (60,  120),
    'right':       (240, 300),
    'back':        (150, 210),
}

# ── Navigation thresholds
──────────────────────────────────────────────────────────────────────────────
```

```
T_FRONT_CLEAR  = 2000   # mm — front path clear (increased: more reaction time)
T_FRONT_CLOSE  = 1200   # mm — front path blocked (increased: stop/turn earlier)
T_SIDE_CLEAR   = 700    # mm — side passage threshold
T_BACK_CLEAR   = 400    # mm — safe to reverse
T_MIN_DIST     = 150    # mm — ignore (rover body / noise)

T_ULTRA_EMRG   = 50     # cm — HC-SR04 emergency distance (stop 50 cm before collision)

T_EDGE_DANGER  = 0.55   # camera edge fill ratio (raised — floor/tape was causing false
triggers)
T_EDGE_CAUTION = 0.35

#genAI Helped finding these numbers
# ── Navigation timing (seconds) ──────────────────────────────────────────────
NAV_COOLDOWN   = 1.5
NAV_STOP_HOLD  = 0.20
NAV_REVERSE_T  = 0.70
NAV_TURN_T     = 0.50

#genAI Helped finding these numbers
# ── Line-follow tuning (pixels / seconds) ────────────────────────────────────
LF_HARD_PX   = 90
LF_GENTLE_PX = 40
LF_GAP_S     = 0.40    # keep going fwd this long after losing line
LF_SEARCH_S  = 1.20   # search-turn this long before giving up

# ═══════════════════════════════════════════════════════════════════════════════
# ═══════════════════════════════════════
#  SHARED STATE
# ═══════════════════════════════════════════════════════════════════════════════
# ═══════════════════════════════════════

class RoverState:
    """All mutable shared state. Use the appropriate lock before writing."""

    def __init__(self):
        # ── Control ──────────────────────────────────────────────────────────
        self.mode     = MODE_MANUAL
        self.estop    = False
        self.last_cmd = 'S'
```

```
        # ── LIDAR ──────────────
```

```
_lidar = None                    # PyRPlidar instance

#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════
#  SERIAL / ARDUINO
#
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════

def _find_arduino():
    """Scan candidate ports, ping Arduino, return connected Serial or None."""
    if not HAS_SERIAL:
        return None
    for port in ARDUINO_PORTS:
        if not os.path.exists(port):
            continue
        try:
            p = serial.Serial(port, SERIAL_BAUD, timeout=1.0)
            time.sleep(2.0)   # wait for Arduino to finish boot/reset
            with S.serial_lock:
                p.reset_input_buffer()
                p.write(b'P')
                resp = p.readline().decode('utf-8', errors='ignore').strip()
            if resp == 'OK':
                print(f"[Serial] Arduino found at {port}")
                return p
            p.close()
        except Exception as e:
            pass
    print("[Serial] Arduino not found — motor commands will be no-ops")
    return None

# Left/Right are physically swapped on this rover (motor wiring reversed).
# Remap here so the fix applies everywhere: manual, auto-nav, line-follow.
_CMD_REMAP = {'L': 'R', 'R': 'L', 'l': 'r', 'r': 'l'}

#genAI Helped for this function
def send_cmd(cmd: str) -> str:
    """
    Send a single-character command to Arduino.
    Acquires serial_lock, flushes input, writes, reads one response line.
    Returns the response string (stripped) or '' on error.
    Respects E-STOP: only 'S' passes through when estop is active.
    Applies _CMD_REMAP to correct physical L/R motor swap.
    """
```

```
    global _ser
    if _ser is None:
        print(f"[CMD] BLOCKED — no serial (_ser is None), cmd={cmd}")
        return ''
    if S.estop and cmd != 'S':
        print(f"[CMD] BLOCKED — ESTOP active, cmd={cmd}")
        return ''
    original = cmd
    cmd = _CMD_REMAP.get(cmd, cmd)   # fix hardware L/R swap
    try:
        with S.serial_lock:
            _ser.reset_input_buffer()
            _ser.write(cmd.encode('utf-8'))
            resp = _ser.readline().decode('utf-8', errors='ignore').strip()
        with S.state_lock:
            S.last_cmd = cmd
        print(f"[CMD] {original}→{cmd}  resp={resp!r}")
        return resp
    except Exception as e:
        print(f"[Serial] send_cmd('{cmd}') error: {e}")
        return ''

#  DISTANCE POLLING THREAD  (HC-SR04 via Arduino)

def distance_thread():
    """Poll HC-SR04 through Arduino every 150 ms. Uses serial_lock."""
    global _ser
    while not S.shutdown.is_set():
        if _ser is not None and not S.estop:
            try:
                with S.serial_lock:
                    _ser.reset_input_buffer()
                    _ser.write(b'D')
                    line = _ser.readline().decode('utf-8', errors='ignore').strip()
                if line.startswith('D:'):
                    raw = line[2:]
                    if raw and raw != '-1':
                        with S.state_lock:
                            S.dist_cm = float(raw)
                            S.dist_ts = time.time()
            except Exception:
                pass
        S.shutdown.wait(timeout=0.15)

#  RPLIDAR THREAD
def _angle_in_sector(angle: float, start: float, end: float) -> bool:
```

```
    """True if angle (0-360) falls in [start, end], handles wrap-around."""
    if start > end:           # sector crosses 0° (e.g. 340-20)
        return angle >= start or angle <= end
    return start <= angle <= end

#genAI Helped generating this function
def lidar_thread():
    """
    Connect to RPLIDAR A1, run start_scan_express(2) continuously,
    update S.lidar_sectors every full revolution.

    CRITICAL: Only start_scan_express(2) works on firmware 1.29 model 24.
    force_scan() and start_scan() cause 'index out of range' errors.
    """
    global _lidar
    if not HAS_LIDAR:
        print("[LIDAR] pyrplidar not available")
        return

    # ── Connect
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
    connected = False
    for port in LIDAR_PORTS:
        if S.shutdown.is_set():
            return
        if not os.path.exists(port):
            continue
        try:
            _lidar = PyRPlidar()
            _lidar.connect(port=port, baudrate=115200, timeout=3)
            time.sleep(0.5)
            try:
                _lidar.stop()    # stop any scan left over from previous session
            except Exception:
                pass
            time.sleep(0.5)
            _lidar.set_motor_pwm(500)
            time.sleep(2)
            info   = _lidar.get_info()
            health = _lidar.get_health()
            fw = getattr(info, 'firmware_version', 'unknown')
            print(f"[LIDAR] Connected on {port} — model={info.model} "
                  f"firmware={fw} health={health.status}")
            with S.state_lock:
                S.lidar_ok = True
```

```
        connected = True
        break
    except Exception as e:
        print(f"[LIDAR] {port}: {e}")
        try:
            _lidar.disconnect()
        except Exception:
            pass
        _lidar = None

if not connected:
    print("[LIDAR] Not available — auto-nav uses camera + ultrasonic only")
    return

# ── Scan loop
──────────────────────────────────────────────────────────────────────────────
# Find a working scan mode. Keep the SAME iterator — never stop/restart
# after getting first data (stop() corrupts the binary protocol with boot text).
active_iter = None
for scan_mode, label in [(2, "express mode 2"), (0, "express mode 0")]:
    if S.shutdown.is_set():
        break
    try:
        print(f"[LIDAR] Trying {label}…")
        gen_fn    = _lidar.start_scan_express(scan_mode)
        candidate = gen_fn()   # start iterating — do NOT call stop() after this
        first     = next(candidate, None)
        if first is not None:
            print(f"[LIDAR] {label} OK — "
                  f"first angle={first.angle:.1f} dist={first.distance:.0f}")
            active_iter = candidate   # keep this iterator, don't restart
            break
        else:
            print(f"[LIDAR] {label} produced no data")
    except Exception as e:
        print(f"[LIDAR] {label} failed: {e}")

if active_iter is None:
    print("[LIDAR] All scan modes failed — LIDAR disabled")
else:
    print("[LIDAR] Scan running…")

try:
    if active_iter is None:
        raise RuntimeError("no working scan mode")
```

```
    rev_buf: dict[float, float] = {}
    meas_count   = 0
    last_hb_time = time.time()

    for meas in active_iter:
        if S.shutdown.is_set():
            break

        meas_count += 1
        now = time.time()
        if now - last_hb_time >= 5.0:
            print(f"[LIDAR] ♥ alive — {meas_count} meas/5s")
            meas_count   = 0
            last_hb_time = now

        angle = meas.angle
        dist  = meas.distance

        if dist < T_MIN_DIST:
            continue

        if meas.start_flag and rev_buf:
            new_sectors: dict[str, float] = {}
            for name, (seg_start, seg_end) in SECTORS.items():
                readings = [
                    rev_buf[a] for a in rev_buf
                    if _angle_in_sector(a, seg_start, seg_end)
                ]
                new_sectors[name] = min(readings) if readings else 9999.0
            with S.state_lock:
                S.lidar_sectors.update(new_sectors)
                S.lidar_ok = True
            rev_buf = {}

        rev_buf[round(angle, 1)] = dist

except Exception as e:
    print(f"[LIDAR] Scan error: {e}")
finally:
    with S.state_lock:
        S.lidar_ok = False
    if _lidar:
        try:
            _lidar.stop()
            _lidar.set_motor_pwm(0)
```

```
            _lidar.disconnect()
        except Exception:
            pass
    print("[LIDAR] Stopped")

#  AUTO NAVIGATION
def _run_maneuver(steps: list):
    """
    Execute a motor maneuver as a sequence of (cmd, sleep_s) pairs.
    Bails early if E-STOP is activated or shutdown is requested.
    """
    for cmd, delay in steps:
        if S.shutdown.is_set() or S.estop:
            send_cmd('S')
            break
        send_cmd(cmd)
        if delay > 0:
            time.sleep(delay)

#genAI Helped generating this function
def _auto_nav_step():
    """
    One navigation decision cycle. Called from nav_thread every 200 ms.
    Reads all sensor data from shared state, then either sends a single
    command (forward/stop) or blocks to execute a multi-step maneuver.
    """
    # ── Snapshot shared state
    with S.state_lock:
        sectors  = dict(S.lidar_sectors)
        dist_cm  = S.dist_cm
        edge_z   = S.edge_zone
        lidar_ok = S.lidar_ok
        nav_ts   = S.nav_ts
        nav_busy = S.nav_busy

    if nav_busy:
        return

    now   = time.time()
    front = sectors['front']
    fl    = sectors['front_left']
    fr    = sectors['front_right']
    left  = sectors['left']
    right = sectors['right']
    back  = sectors['back']
```

```
def begin_maneuver():
    with S.state_lock:
        S.nav_busy = True
        S.nav_ts   = time.time()

def end_maneuver():
    with S.state_lock:
        S.nav_busy = False

# ── Priority 1: HC-SR04 emergency ──────────────────────────────────────
if dist_cm > 0 and dist_cm < T_ULTRA_EMRG:
    if now - nav_ts < NAV_COOLDOWN:
        return
    begin_maneuver()
    turn = 'R' if right > left else 'L'
    steps = [('S', NAV_STOP_HOLD)]
    if back > T_BACK_CLEAR:
        steps.append(('b', NAV_REVERSE_T))
    steps += [(turn, NAV_TURN_T), ('S', 0)]
    _run_maneuver(steps)
    end_maneuver()
    return

# ── Priority 2: Camera danger zone ─────────────────────────────────────
if edge_z == 'DANGER':
    if now - nav_ts < NAV_COOLDOWN:
        return
    begin_maneuver()
    turn = 'R' if right > left else 'L'
    steps = [('S', NAV_STOP_HOLD)]
    if back > T_BACK_CLEAR:
        steps.append(('b', NAV_REVERSE_T))
    steps += [(turn, NAV_TURN_T), ('S', 0)]
    _run_maneuver(steps)
    end_maneuver()
    return

# ── No LIDAR: use camera caution only ──────────────────────────────────
if not lidar_ok:
    send_cmd('S' if edge_z == 'CAUTION' else 'F')
    return
```

```
    # ── LIDAR-based navigation ──────────────────────────────────────────
    if front > T_FRONT_CLEAR:
        # Path is clear — drive forward
        send_cmd('F')
        return

    if front > T_FRONT_CLOSE:
        # Approaching — continue forward (cooldown prevents jitter)
        send_cmd('F')
        return

    # ── Front blocked ──────────────────────────────────────────────────────
    if now - nav_ts < NAV_COOLDOWN:
        return   # respect cooldown before another maneuver

    begin_maneuver()

    if fl < T_SIDE_CLEAR and fr < T_SIDE_CLEAR:
        # Boxed in on front and both flanks
        if back > T_BACK_CLEAR:
            # Reverse then turn toward most open side
            turn = 'R' if right > left else 'L'
            steps = [('S', NAV_STOP_HOLD),
                     ('b', NAV_REVERSE_T),
                     (turn, NAV_TURN_T),
                     ('S', 0)]
        else:
            # Completely boxed in — just stop
            steps = [('S', 0)]
    else:
        # At least one flank is open — turn toward it
        turn = 'R' if fr >= fl else 'L'
        steps = [('S', NAV_STOP_HOLD),
                 (turn, NAV_TURN_T),
                 ('S', 0)]

    _run_maneuver(steps)
    end_maneuver()

def nav_thread():
    """Auto-nav decision loop — runs only when mode is MODE_AUTO."""
    while not S.shutdown.is_set():
        if S.mode == MODE_AUTO and not S.estop:
            try:
```

```
            _auto_nav_step()
        except Exception as e:
            print(f"[Nav] Error in decision step: {e}")
    S.shutdown.wait(timeout=0.1)

#  CAMERA THREAD
def _make_placeholder_jpeg() -> bytes | None:
    """Generate a 'NO CAMERA' placeholder JPEG."""
    if not HAS_CV:
        return None
    img = np.zeros((480, 640, 3), dtype=np.uint8)
    cv2.putText(img, "NO CAMERA", (190, 230),
                cv2.FONT_HERSHEY_SIMPLEX, 1.6, (70, 70, 70), 2)
    cv2.putText(img, "Check USB connection", (175, 275),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (50, 50, 50), 1)
    _, buf = cv2.imencode('.jpg', img)
    return buf.tobytes()

def _draw_backup_grid(frame, h: int, w: int):
    """
    Draw backup-camera style guide overlay:
      red zone line  at 75% height
      yellow zone    at 55%
      green zone     at 40%
      converging guide lines from center-bottom
    """
    cx      = w // 2
    y_red   = int(h * 0.75)
    y_yel   = int(h * 0.55)
    y_grn   = int(h * 0.40)

    cv2.line(frame, (0, y_red), (w, y_red), (0, 0, 255),   1)
    cv2.line(frame, (0, y_yel), (w, y_yel), (0, 220, 220), 1)
    cv2.line(frame, (0, y_grn), (w, y_grn), (0, 255, 0),   1)

    # Two pairs of converging guide lines
    cv2.line(frame, (cx, h), (0,          y_red), (0, 180, 180), 1)
    cv2.line(frame, (cx, h), (w,          y_red), (0, 180, 180), 1)
    cv2.line(frame, (cx, h), (int(w*.2), y_grn),  (0, 180, 180), 1)
    cv2.line(frame, (cx, h), (int(w*.8), y_grn),  (0, 180, 180), 1)
    return frame

def _process_edge_detection(raw_frame, display, h: int, w: int):
    """
    Canny edge detection on the bottom 25% of the frame (danger zone).
    Updates S.edge_fill and S.edge_zone.
```

```
    Overlays a coloured edge map on the display frame.
    """
    y_danger = int(h * 0.75)
    roi = raw_frame[y_danger:, :]

    gray    = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges   = cv2.Canny(blurred, 50, 150)

    total = edges.size
    fill  = float(np.count_nonzero(edges)) / total if total > 0 else 0.0

    if fill > T_EDGE_DANGER:
        zone = 'DANGER'
    elif fill > T_EDGE_CAUTION:
        zone = 'CAUTION'
    else:
        zone = 'OK'

    with S.state_lock:
        S.edge_fill = fill
        S.edge_zone = zone

    # Colour-code edge overlay
    e3 = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
    if zone == 'DANGER':
        coloured = e3 * np.array([0, 0, 1], dtype=np.uint8)
    elif zone == 'CAUTION':
        coloured = e3 * np.array([0, 1, 1], dtype=np.uint8)
    else:
        coloured = e3 * np.array([0, 1, 0], dtype=np.uint8)

    display[y_danger:, :] = cv2.addWeighted(
        display[y_danger:, :], 0.70, coloured, 0.30, 0)

    label_col = {'DANGER': (0, 0, 255), 'CAUTION': (0, 220, 220),
                 'OK': (0, 255, 0)}[zone]
    cv2.putText(display, f"CAM:{zone} {fill:.0%}",
                (4, h - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.45, label_col, 1)
    return display

def _lf_safe_fwd():
    """Send F only if HC-SR04 distance is safe (>50cm). Otherwise stop."""
    with S.state_lock:
        dist = S.dist_cm
    if dist > 0 and dist < T_ULTRA_EMRG:
```

```
        send_cmd('S')
        return False   # blocked
    send_cmd('F')
    return True

#genAI helped in this function
def _process_line_follow(raw_frame, display, h: int, w: int):
    """
    Line follow — black electrical tape on a light floor.

    Detection uses Otsu adaptive threshold (handles any lighting automatically)
    combined with an HSV darkness check to reject coloured objects.
    A contour must pass area + width + aspect-ratio checks to be accepted as tape.

    Collision safety: HC-SR04 is checked before every forward command.
    When line is lost the rover STOPS immediately — it does NOT drive blind.
    """
    roi_y = int(h * 0.80)   # bottom 20% only — stays within red danger zone,
    roi   = raw_frame[roi_y:, :]  # avoids detecting distant objects/furniture

    # ── Step 1: Otsu threshold on grayscale ──────────────────────────────────
    # Light blur only — heavy blur erases thin tape lines.
    # Otsu auto-finds the threshold between dark tape and bright floor.
    blurred = cv2.GaussianBlur(roi, (5, 5), 0)
    gray    = cv2.cvtColor(blurred, cv2.COLOR_BGR2GRAY)
    _, otsu = cv2.threshold(gray, 0, 255,
                            cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # ── Step 2: HSV darkness gate — reject coloured objects ──────────────────
    hsv  = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
    dark = cv2.inRange(hsv,
                       np.array([0,   0,   0]),
                       np.array([180, 255, 90]))  # Value < 90 = dark/black pixel

    # ── Step 3: Combine both masks ───────────────────────────────────────────
    mask = cv2.bitwise_and(otsu, dark)

    # ── Step 4: Morphological cleanup ────────────────────────────────────────
    # IMPORTANT: use small 3×3 kernel for OPEN so thin tape lines are not erased.
    # A 7×7 open requires a 7×7 solid block to survive — a 15px wide tape line fails that.
    k_open  = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    k_close = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    mask    = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  k_open)   # remove tiny noise dots
```

```
mask    = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k_close)  # fill small gaps in tape

# ── ROI overlay: cyan = detected tape ──────────────────────────────────────
overlay = np.zeros_like(display[roi_y:, :])
overlay[mask > 0] = (255, 255, 0)
display[roi_y:, :] = cv2.addWeighted(display[roi_y:, :], 0.6, overlay, 0.4, 0)
cv2.rectangle(display, (0, roi_y), (w - 1, h - 1), (255, 0, 255), 2)

# ── Step 5: Contour detection + shape validation ──────────────────────────
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cx_frame = w // 2
now      = time.time()

valid_cnts = []
for c in cnts:
    area = cv2.contourArea(c)
    if area < 200:
        continue         # too small — noise dot

    if area > 12000:
        continue         # too large — chair base, table leg base, large shadow
                         # tape at any angle/distance fits within this limit

    _x, _y, cw, _ch = cv2.boundingRect(c)
    if cw < 10:
        continue         # too narrow — single-pixel edge artifact

    # ── Elongation check using minimum-area rotated rectangle ──────────
    # Works for diagonal tape: finds the true long/short axes regardless of angle.
    # Tape (long thin stripe): ratio >> 1.8
    # Circular chair base:     ratio ≈ 1.0  → rejected
    # Square/blob objects:     ratio ≈ 1.0  → rejected
    if len(c) >= 5:       # minAreaRect needs at least 5 points
        _, (rw, rh), _ = cv2.minAreaRect(c)
        long_side  = max(rw, rh)
        short_side = min(rw, rh)
        if short_side > 0 and long_side / short_side < 1.8:
            continue      # not elongated enough to be tape

    valid_cnts.append(c)

if valid_cnts:
    largest = max(valid_cnts, key=cv2.contourArea)
    M = cv2.moments(largest)
    if M['m00'] > 0:
```

```
        lx = int(M['m10'] / M['m00'])
        ly = int(M['m01'] / M['m00']) + roi_y

        cv2.drawContours(display[roi_y:, :], [largest], -1, (0, 255, 0), 2)
        cv2.circle(display, (lx, ly), 8, (0, 100, 255), -1)
        cv2.line(display, (cx_frame, h - 1), (lx, ly), (255, 220, 0), 2)

        error = lx - cx_frame
        cv2.putText(display, f"err:{error:+d}px", (4, roi_y + 18),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 220, 0), 1)

        with S.state_lock:
            S.lf_last_seen = now
            S.lf_dir = ('left'  if error < -LF_GENTLE_PX else
                        'right' if error >  LF_GENTLE_PX else 'center')

        if not S.estop:
            ae = abs(error)
            if   ae < LF_GENTLE_PX: _lf_safe_fwd()                    # straight
            elif ae < LF_HARD_PX:   send_cmd('l' if error < 0 else 'r')  # gentle curve
            else:                   send_cmd('L' if error < 0 else 'R')  # hard pivot
        return display

    # ── Line not found
    ────────────────────────────────────────────────────────────────────────────
    with S.state_lock:
        lost_s = now - S.lf_last_seen
        lf_dir = S.lf_dir

    cv2.putText(display, f"LINE LOST {lost_s:.1f}s",
                (cx_frame - 65, roi_y + 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 60, 255), 2)

    if not S.estop:
        if lost_s < 0.25:
            send_cmd('S')                       # stop immediately — don't drive blind
        elif lost_s < LF_SEARCH_S:
            send_cmd('L' if lf_dir == 'left' else 'R')  # slow search toward last direction
        else:
            send_cmd('S')                       # give up completely

    return display

def camera_thread():
    """
    Camera capture, frame processing, JPEG encoding.
```

```
Writes JPEG bytes to S.frame_buf (protected by S.frame_lock).
Drives line-follow commands when in MODE_LINE.
"""
if not HAS_CV:
    print("[Camera] OpenCV not available")
    return

cap = None
for idx in [CAMERA_IDX, 1 - CAMERA_IDX]:
    cap = cv2.VideoCapture(idx)
    if cap.isOpened():
        cap.set(cv2.CAP_PROP_FRAME_WIDTH,  640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        cap.set(cv2.CAP_PROP_FPS, 30)
        print(f"[Camera] Opened index {idx}")
        with S.state_lock:
            S.cam_ok = True
        break
    cap.release()
    cap = None

if cap is None:
    print("[Camera] No camera found")
    placeholder = _make_placeholder_jpeg()
    with S.frame_lock:
        S.frame_buf = placeholder
    return

while not S.shutdown.is_set():
    ret, frame = cap.read()
    if not ret:
        S.shutdown.wait(timeout=0.05)
        continue

    h, w = frame.shape[:2]
    display = frame.copy()
    mode    = S.mode

    # ── Per-mode processing
```

```
    if mode == MODE_LINE and not S.estop:
        display = _process_line_follow(frame, display, h, w)
    elif mode == MODE_AUTO and not S.estop:
        display = _draw_backup_grid(display, h, w)
        display = _process_edge_detection(frame, display, h, w)
    else:
```

```
        display = _draw_backup_grid(display, h, w)

    # ── Mode label ──────────────────────────────────────────────────────────
    mode_labels = {MODE_MANUAL: 'MANUAL',
                   MODE_AUTO:   'AUTO NAV',
                   MODE_LINE:   'LINE FOLLOW'}
    label = 'EMERGENCY STOP' if S.estop else mode_labels.get(mode, mode.upper())
    lcolor = (0, 0, 200) if S.estop else (230, 230, 230)
    # Background pill
    (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cv2.rectangle(display, (2, 4), (tw + 10, th + 12), (0, 0, 0), -1)
    cv2.putText(display, label, (6, th + 6),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, lcolor, 2)

    # ── Recording ───────────────────────────────────────────────────────────
    if S.recording:
        if S.rec_writer is None:
            ts     = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
            path   = f'/tmp/rover_{ts}.avi'
            fourcc = cv2.VideoWriter_fourcc(*'XVID')
            S.rec_writer = cv2.VideoWriter(path, fourcc, 15.0, (w, h))
            S.rec_path   = path
            print(f"[Camera] Recording → {path}")
        if S.rec_writer:
            S.rec_writer.write(display)
        # REC badge
        cv2.circle(display, (w - 18, 14), 8, (0, 0, 220), -1)
        cv2.putText(display, 'REC', (w - 54, 18),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 0, 220), 1)
    elif S.rec_writer is not None:
        S.rec_writer.release()
        S.rec_writer = None
        print(f"[Camera] Recording saved → {S.rec_path}")

    # ── Encode and store ────────────────────────────────────────────────────
    ok, buf = cv2.imencode('.jpg', display, [cv2.IMWRITE_JPEG_QUALITY, 72])
    if ok:
        with S.frame_lock:
            S.frame_buf = buf.tobytes()

    S.shutdown.wait(timeout=0.033)   # ~30 fps target

cap.release()
```

```
    if S.rec_writer:
        S.rec_writer.release()
    print("[Camera] Stopped")

# HTTP SERVER

# Embedded dashboard HTML
DASHBOARD_HTML = """\
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Rover Dashboard</title>
<style>
*{box-sizing:border-box;margin:0;padding:0}
body{background:#0d1117;color:#c9d1d9;font-family:'Courier New',monospace;font-size:13px;
     overflow-x:hidden;user-select:none}

/* ── E-Stop banner
──────────────────────────────────────────────────────────────────────────────── */
#estop-banner{display:none;text-align:center;padding:10px;font-size:18px;
  font-weight:bold;cursor:pointer;letter-spacing:2px;
  background:#7f1d1d;animation:pulse-red 1s ease-in-out infinite}
#estop-banner.active{display:block}
@keyframes pulse-red{0%,100%{background:#7f1d1d}50%{background:#dc2626}}

/* ── Top bar
──────────────────────────────────────────────────────────────────────────────── */
.topbar{display:flex;align-items:center;background:#161b22;padding:8px 14px;
  gap:12px;border-bottom:2px solid #21262d}
.topbar h1{color:#f85149;font-size:16px;letter-spacing:1px;flex:1}
.topbar-status{font-size:11px;color:#6e7681}
.topbar-status span{color:#58a6ff}

/* ── Layout
──────────────────────────────────────────────────────────────────────────────
*/
.main{display:grid;grid-template-columns:1fr 310px;gap:10px;
  padding:10px;height:calc(100vh - 88px);min-height:0}

/* ── Camera panel
──────────────────────────────────────────────────────────────────────────────── */
.cam-panel{background:#000;border-radius:8px;overflow:hidden;position:relative;
  display:flex;align-items:center;justify-content:center;
  border:2px solid #21262d;min-height:200px}
```

```
.cam-panel img{max-width:100%;max-height:100%;object-fit:contain;display:block}
.cam-badge{position:absolute;top:6px;right:8px;background:rgba(0,0,0,.65);
  padding:3px 8px;border-radius:4px;font-size:10px;color:#58a6ff}
.rec-dot{color:#f85149;animation:blink 1s infinite}
@keyframes blink{50%{opacity:0}}

/* ── Right column
──────────────────────────────────────────────────────────────────────────────── */
.right-col{display:flex;flex-direction:column;gap:8px;overflow-y:auto}
.panel{background:#161b22;border-radius:8px;padding:10px;
  border:1px solid #21262d;flex-shrink:0}
.panel-title{color:#f85149;font-size:10px;font-weight:bold;
  text-transform:uppercase;letter-spacing:1px;margin-bottom:8px}

/* ── Mode buttons
──────────────────────────────────────────────────────────────────────────────── */
.mode-row{display:flex;gap:6px}
.btn-mode{flex:1;padding:8px 4px;border:2px solid #30363d;background:#0d1117;
  color:#8b949e;border-radius:6px;cursor:pointer;font-size:11px;
  font-family:inherit;font-weight:bold;letter-spacing:.5px;transition:all .15s}
.btn-mode:hover{border-color:#58a6ff;color:#c9d1d9}
.btn-mode.active{background:#1f6feb;border-color:#58a6ff;color:#fff}

/* ── Drive grid
──────────────────────────────────────────────────────────────────────────────── */
.drive-grid{display:grid;grid-template-columns:repeat(3,1fr);
  grid-template-rows:repeat(3,1fr);gap:5px;aspect-ratio:1;max-height:180px}
.btn-drive{display:flex;align-items:center;justify-content:center;
  background:#0d1117;border:2px solid #30363d;border-radius:8px;
  color:#8b949e;font-size:22px;cursor:pointer;transition:all .1s;
  touch-action:manipulation;-webkit-tap-highlight-color:transparent;
  min-height:44px}
.btn-drive:active,.btn-drive.pressed{background:#1f3a5f;border-color:#58a6ff;
  color:#fff;transform:scale(.94)}
.btn-stop{background:#1c0a0a!important;border-color:#da3633!important;
  color:#f85149!important;font-size:11px!important;font-weight:bold}
.btn-stop:active{background:#5a1a1a!important}
.cell-empty{border:none;background:transparent;pointer-events:none}

/* ── LIDAR sector grid
──────────────────────────────────────────────────────────────────────────────── */
.sector-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:5px}
.sector-box{text-align:center;padding:6px 3px;border-radius:6px;
  border:1px solid #30363d;background:#0d1117}
.sector-box.ok    {border-color:#1a3a1a}
.sector-box.warn  {border-color:#3a3010}
```

```
.sector-box.danger{border-color:#3a1010}
.sec-label{font-size:9px;color:#6e7681;text-transform:uppercase}
.sec-dist {font-size:15px;font-weight:bold;margin:2px 0;line-height:1}
.sec-dist.ok    {color:#3fb950}
.sec-dist.warn  {color:#d29922}
.sec-dist.danger{color:#f85149}

/* ── Sensor rows
──────────────────────────────────────────────────────────────────────────────── */
.sensor-row{display:flex;justify-content:space-between;align-items:center;
  padding:5px 0;border-bottom:1px solid #21262d}
.sensor-row:last-child{border:none}
.sensor-label{color:#6e7681;font-size:11px}
.sensor-val  {font-size:14px;font-weight:bold}
.sv-ok     {color:#3fb950}
.sv-warn   {color:#d29922}
.sv-danger {color:#f85149}
.sv-neutral{color:#8b949e}

/* ── Record button
──────────────────────────────────────────────────────────────────────────────── */
.btn-record{width:100%;padding:7px;background:#0d1117;border:2px solid #3a1010;
  color:#f85149;border-radius:6px;cursor:pointer;font-family:inherit;
  font-size:12px;font-weight:bold;transition:all .2s}
.btn-record.active{background:#3a1010;border-color:#f85149;
  animation:pulse-red 1s infinite}
.btn-record:hover{border-color:#f85149}

/* ── Emergency stop
──────────────────────────────────────────────────────────────────────────────── */
.btn-estop{width:100%;padding:14px;background:#3a1010;border:3px solid #f85149;
  color:#fff;border-radius:8px;cursor:pointer;font-family:inherit;font-size:15px;
  font-weight:bold;letter-spacing:2px;animation:pulse-red 2.5s infinite}
.btn-estop.resumed{background:#0d2818;border-color:#3fb950;color:#3fb950;
  animation:none;letter-spacing:.5px}

/* ── Status bar
──────────────────────────────────────────────────────────────────────────────── */
.statusbar{position:fixed;bottom:0;left:0;right:0;background:#161b22;
  border-top:1px solid #21262d;padding:3px 14px;display:flex;gap:16px;
  font-size:10px;color:#6e7681}
.statusbar span{color:#58a6ff}
</style>
</head>
<body>
```

```
<div id="estop-banner" onclick="toggleEstop()">
  ⚡ EMERGENCY STOP ACTIVE — TAP TO RESUME ⚡
</div>

<div class="topbar">
  <h1>ROVER DASHBOARD</h1>
  <div class="topbar-status">
    Mode: <span id="ts-mode">MANUAL</span>  | 
    Cmd:  <span id="ts-cmd">S</span>
  </div>
</div>

<div class="main">

  <!-- ── Left: Camera
──────────────────────────────────────────────────────────────────────────── -->
  <div class="cam-panel">
    <img id="cam-img" src="" alt="camera">
    <div class="cam-badge">
      <span id="rec-dot" class="rec-dot" style="display:none">● </span>640×480
    </div>
  </div>

  <!-- ── Right: Controls ──────────────────────────────────────────────────────────
-->
  <div class="right-col">

    <!-- Mode selection -->
    <div class="panel">
      <div class="panel-title">Mode</div>
      <div class="mode-row">
        <button class="btn-mode active" id="bm-
manual"     onclick="setMode('manual')">MANUAL</button>
        <button class="btn-mode"        id="bm-auto_nav"   onclick="setMode('auto_nav')">AUTO
NAV</button>
        <button class="btn-mode"        id="bm-line_follow"
onclick="setMode('line_follow')">LINE FOLLOW</button>
      </div>
    </div>

    <!-- Drive controls -->
    <div class="panel">
      <div class="panel-title">Drive — WASD / Arrows / Mouse</div>
      <div class="drive-grid">
        <div class="btn-drive cell-empty"></div>
        <div class="btn-drive" id="bd-F"
```

```
        onmousedown="beginDrive('F')" ontouchstart="beginDrive('F')"
        onmouseup="endDrive()"       ontouchend="endDrive()">▲</div>
      <div class="btn-drive cell-empty"></div>

      <div class="btn-drive" id="bd-L"
        onmousedown="beginDrive('L')" ontouchstart="beginDrive('L')"
        onmouseup="endDrive()"       ontouchend="endDrive()">◀</div>
      <div class="btn-drive btn-stop"
        onclick="driveNow('S')">STOP</div>
      <div class="btn-drive" id="bd-R"
        onmousedown="beginDrive('R')" ontouchstart="beginDrive('R')"
        onmouseup="endDrive()"       ontouchend="endDrive()">▶</div>

      <div class="btn-drive cell-empty"></div>
      <div class="btn-drive" id="bd-b"
        onmousedown="beginDrive('b')" ontouchstart="beginDrive('b')"
        onmouseup="endDrive()"       ontouchend="endDrive()">▼</div>
      <div class="btn-drive cell-empty"></div>
    </div>
  </div>

  <!-- LIDAR sectors -->
  <div class="panel">
    <div class="panel-title">LIDAR Sectors
      <span id="lidar-badge" style="color:#6e7681;font-size:9px;margin-
left:4px">(offline)</span>
    </div>
    <div class="sector-grid">
      <!-- Row 1: FL, F, FR -->
      <div class="sector-box" id="sb-front_left">
        <div class="sec-label">FRONT-L</div>
        <div class="sec-dist" id="sd-front_left">---</div>
      </div>
      <div class="sector-box" id="sb-front">
        <div class="sec-label">FRONT</div>
        <div class="sec-dist" id="sd-front">---</div>
      </div>
      <div class="sector-box" id="sb-front_right">
        <div class="sec-label">FRONT-R</div>
        <div class="sec-dist" id="sd-front_right">---</div>
      </div>
      <!-- Row 2: L, B, R -->
      <div class="sector-box" id="sb-left">
        <div class="sec-label">LEFT</div>
        <div class="sec-dist" id="sd-left">---</div>
      </div>
```

```
      <div class="sector-box" id="sb-back">
        <div class="sec-label">BACK</div>
        <div class="sec-dist" id="sd-back">---</div>
      </div>
      <div class="sector-box" id="sb-right">
        <div class="sec-label">RIGHT</div>
        <div class="sec-dist" id="sd-right">---</div>
      </div>
    </div>
  </div>

  <!-- Sensors -->
  <div class="panel">
    <div class="panel-title">Sensors</div>
    <div class="sensor-row">
      <div class="sensor-label">HC-SR04 Distance</div>
      <div class="sensor-val sv-neutral" id="sv-ultra">-- cm</div>
    </div>
    <div class="sensor-row">
      <div class="sensor-label">Camera Zone</div>
      <div class="sensor-val sv-neutral" id="sv-zone">--</div>
    </div>
    <div class="sensor-row">
      <div class="sensor-label">Edge Fill</div>
      <div class="sensor-val sv-neutral" id="sv-fill">--%</div>
    </div>
  </div>

  <!-- Recording -->
  <div class="panel">
    <div class="panel-title">Recording</div>
    <button class="btn-record" id="btn-rec" onclick="toggleRecord()">
      ● START RECORDING
    </button>
  </div>

  <!-- Emergency Stop -->
  <div class="panel">
    <button class="btn-estop" id="btn-estop" onclick="toggleEstop()">
      ⚡ EMERGENCY STOP ⚡
    </button>
  </div>

  </div><!-- .right-col -->
</div><!-- .main -->
```

```
<div class="statusbar">
  <div>Mode: <span id="sb-mode">MANUAL</span></div>
  <div>Cmd:  <span id="sb-cmd">S</span></div>
  <div>LIDAR:<span id="sb-lidar">offline</span></div>
  <div>HC-SR04:<span id="sb-dist">--</span></div>
  <div>Cam:<span id="sb-cam">--</span></div>
</div>

<script>
// ── State
──────────────
```

```
  // Auto-stop 200 ms after button release (safety)
  releaseTimer = setTimeout(() => {
    if (currentMode === 'manual' && !estopActive) driveNow('S');
  }, 200);
}

// ── Mode ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```
function setMode(mode) {
  fetch('/mode', {
    method: 'POST',
    headers: {'Content-Type':'application/json'},
    body: JSON.stringify({mode})
  }).then(r => r.json()).then(d => {
    if (d.ok) syncModeButtons(mode);
  });
}

function syncModeButtons(mode) {
  currentMode = mode;
  document.querySelectorAll('.btn-mode').forEach(b => b.classList.remove('active'));
  const btn = document.getElementById('bm-' + mode);
  if (btn) btn.classList.add('active');
}

// ── E-Stop ──
```
```

```
  } else {
    banner.classList.remove('active');
    btn.classList.add('resumed');
    btn.textContent = '✓ OPERATION RESUMED — tap to re-arm stop';
  }
}

// ── Recording ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
function toggleRecord() {
  recording = !recording;
  fetch('/record', {
    method: 'POST',
    headers: {'Content-Type':'application/json'},
    body: JSON.stringify({recording})
  });
  const btn = document.getElementById('btn-rec');
  const dot = document.getElementById('rec-dot');
  if (recording) {
    btn.textContent = '■ STOP RECORDING';
    btn.classList.add('active');
    dot.style.display = 'inline';
  } else {
    btn.textContent = '● START RECORDING';
    btn.classList.remove('active');
    dot.style.display = 'none';
  }
}

// ── Keyboard ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
const KEY_MAP = {
  ArrowUp:'F', w:'F', W:'F',
  ArrowDown:'b', s:'b', S:'b',
  ArrowLeft:'L', a:'L', A:'L',
  ArrowRight:'R', d:'R', D:'R',
  ' ':'S'
};
const keysHeld = new Set();

document.addEventListener('keydown', e => {
  if (keysHeld.has(e.key)) return;
  if (e.key === ' ') e.preventDefault();
  keysHeld.add(e.key);
```

```
  const cmd = KEY_MAP[e.key];
  if (cmd) { beginDrive(cmd); }
});
document.addEventListener('keyup', e => {
  keysHeld.delete(e.key);
  if (KEY_MAP[e.key]) endDrive();
});

// ── Status polling ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

```
      const dist = document.getElementById('sd-' + name);
      if (!box || !dist) continue;
      const cls = sectorClass(mm);
      box.className  = 'sector-box ' + cls;
      dist.className = 'sec-dist ' + cls;
      dist.textContent = sectorLabel(mm);
    }

    // Ultrasonic
    const cm = d.dist_cm;
    const ultraEl = document.getElementById('sv-ultra');
    ultraEl.textContent = cm > 0 ? cm.toFixed(1) + ' cm' : '-- cm';
    ultraEl.className = 'sensor-val ' +
      (cm > 0 && cm < 18 ? 'sv-danger' : cm > 0 && cm < 40 ? 'sv-warn' : 'sv-ok');
    document.getElementById('sb-dist').textContent = cm > 0 ? cm.toFixed(1) + 'cm' : '--';

    // Camera zone
    const zone   = d.edge_zone || 'OK';
    const zoneEl = document.getElementById('sv-zone');
    zoneEl.textContent = zone;
    zoneEl.className = 'sensor-val ' +
      (zone === 'DANGER' ? 'sv-danger' : zone === 'CAUTION' ? 'sv-warn' : 'sv-ok');
    document.getElementById('sv-fill').textContent =
      ((d.edge_fill || 0) * 100).toFixed(1) + '%';
    document.getElementById('sb-cam').textContent = zone;
  })
  .catch(() => {});
}

// ── Camera frame polling ──────────────────────────────────────────────────────
// We poll /frame?ts instead of using MJPEG src directly to handle
// reconnections cleanly and avoid browser MJPEG buffering issues.
let lastFrameTs = 0;
function refreshFrame() {
  const img = document.getElementById('cam-img');
  const url = '/frame?' + Date.now();
  const tmp = new Image();
  tmp.onload = () => {
    img.src = tmp.src;
    lastFrameTs = Date.now();
  };
  tmp.onerror = () => {};
  tmp.src = url;
}
```

```
setInterval(updateStatus, 500);
setInterval(refreshFrame,  100);

// Bootstrap
updateStatus();
refreshFrame();
</script>
</body>
</html>
"""

# ── HTTP handler ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

class RoverHandler(BaseHTTPRequestHandler):
    """Handles all HTTP requests from the dashboard."""

    def log_message(self, fmt, *args):
        pass   # suppress default per-request logging

    # ── GET ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

    def do_GET(self):
        path = self.path.split('?')[0]

        if path == '/':
            self._send_html(DASHBOARD_HTML)

        elif path == '/frame':
            # Single JPEG frame for JS polling
            with S.frame_lock:
                data = S.frame_buf
            if data:
                self._send_bytes(data, 'image/jpeg',
                                 extra=[('Cache-Control', 'no-cache, no-store')])
            else:
                self.send_response(503)
                self.end_headers()

        elif path == '/stream':
            # MJPEG stream (kept as fallback / direct use)
            self.send_response(200)
            self.send_header('Content-Type',
                             'multipart/x-mixed-replace; boundary=frame')
```

```
        self.send_header('Cache-Control', 'no-cache')
        self.send_header('Connection', 'keep-alive')
        self.end_headers()
        try:
            while not S.shutdown.is_set():
                with S.frame_lock:
                    data = S.frame_buf
                if data:
                    self.wfile.write(b'--frame\r\n')
                    self.wfile.write(b'Content-Type: image/jpeg\r\n')
                    self.wfile.write(
                        f'Content-Length: {len(data)}\r\n\r\n'.encode())
                    self.wfile.write(data)
                    self.wfile.write(b'\r\n')
                    self.wfile.flush()
                time.sleep(0.033)
        except (BrokenPipeError, ConnectionResetError):
            pass

    elif path == '/status':
        with S.state_lock:
            payload = {
                'mode':          S.mode,
                'estop':         S.estop,
                'recording':     S.recording,
                'last_cmd':      S.last_cmd,
                'lidar_ok':      S.lidar_ok,
                'lidar_sectors': dict(S.lidar_sectors),
                'dist_cm':       S.dist_cm,
                'cam_ok':        S.cam_ok,
                'edge_fill':     S.edge_fill,
                'edge_zone':     S.edge_zone,
            }
        self._send_json(payload)

    else:
        self.send_response(404)
        self.end_headers()

# ── POST ──────────────
```

```
        data = json.loads(body)
    except Exception:
        data = {}

    path = self.path.split('?')[0]

    if path == '/command':
        cmd = str(data.get('cmd', ''))
        if len(cmd) == 1 and not S.estop:
            if S.mode == MODE_MANUAL or cmd == 'S':
                send_cmd(cmd)
        self._send_json({'ok': True})

    elif path == '/mode':
        mode = data.get('mode', MODE_MANUAL)
        if mode in (MODE_MANUAL, MODE_AUTO, MODE_LINE):
            with S.state_lock:
                S.mode = mode
            send_cmd('S')   # safe stop on mode switch
        self._send_json({'ok': True, 'mode': S.mode})

    elif path == '/estop':
        active = bool(data.get('active', True))
        with S.state_lock:
            S.estop = active
        if active:
            send_cmd('S')
        self._send_json({'ok': True, 'estop': active})

    elif path == '/record':
        rec = bool(data.get('recording', False))
        with S.state_lock:
            S.recording = rec
            if not rec and S.rec_writer is not None:
                S.rec_writer.release()
                S.rec_writer = None
        self._send_json({'ok': True, 'recording': rec})

    else:
        self.send_response(404)
        self.end_headers()

# ── Helpers ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

```
    def _send_html(self, html: str):
```

```
        body = html.encode('utf-8')
        self.send_response(200)
        self.send_header('Content-Type', 'text/html; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _send_json(self, obj):
        body = json.dumps(obj).encode('utf-8')
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _send_bytes(self, data: bytes, mime: str, extra=None):
        try:
            self.send_response(200)
            self.send_header('Content-Type', mime)
            self.send_header('Content-Length', str(len(data)))
            if extra:
                for k, v in extra:
                    self.send_header(k, v)
            self.end_headers()
            self.wfile.write(data)
        except (BrokenPipeError, ConnectionResetError):
            pass   # browser closed connection before frame was fully sent — normal

class ThreadedHTTPServer(ThreadingMixIn, HTTPServer):
    """One thread per connection — needed for MJPEG stream + concurrent API."""
    daemon_threads = True
    allow_reuse_address = True

    def handle_error(self, request, client_address):
        """Silence BrokenPipe — these are normal with 100 ms frame polling."""
        import sys
        exc = sys.exc_info()[1]
        if isinstance(exc, (BrokenPipeError, ConnectionResetError)):
            return
        super().handle_error(request, client_address)


#  STARTUP / SHUTDOWN


def _shutdown(sig=None, frame=None):
```

```
    print("\n[Rover] Shutting down…")
    S.shutdown.set()

    # Stop motors
    if _ser:
        try:
            with S.serial_lock:
                _ser.write(b'S')
            time.sleep(0.15)
            _ser.close()
        except Exception:
            pass

    # Stop LIDAR
    if _lidar and S.lidar_ok:
        try:
            _lidar.stop()
            _lidar.set_motor_pwm(0)
            _lidar.disconnect()
        except Exception:
            pass

    # Stop recording
    with S.state_lock:
        if S.rec_writer:
            S.rec_writer.release()
            S.rec_writer = None

    print("[Rover] All systems stopped. Goodbye.")
    sys.exit(0)

def _get_local_ip() -> str:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(('8.8.8.8', 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except Exception:
        return '127.0.0.1'

def main():
    global _ser

    print("=" * 62)
    print("  ROVER DASHBOARD  —  Autonomous Delivery Rover")
```

```
    print("=" * 62)
    print()

    # Arduino
    print("[Serial] Searching for Arduino…")
    _ser = _find_arduino()

    # Background threads
    thread_specs = [
        ('lidar',    lidar_thread),
        ('camera',   camera_thread),
        ('distance', distance_thread),
        ('nav',      nav_thread),
    ]
    for name, target in thread_specs:
        t = threading.Thread(target=target, name=name, daemon=True)
        t.start()
        print(f"[Main]   Thread '{name}' started")

    #  Signal handlers
    signal.signal(signal.SIGINT,  _shutdown)
    signal.signal(signal.SIGTERM, _shutdown)

    # ── HTTP server
    server = ThreadedHTTPServer(('0.0.0.0', HTTP_PORT), RoverHandler)
    local_ip = _get_local_ip()

    print()
    print(f"[Dashboard] http://{local_ip}:{HTTP_PORT}      ← open this on any device")
    print(f"[Dashboard] http://localhost:{HTTP_PORT}")
    print("[Dashboard] Press Ctrl+C to stop")
    print()

   server.serve_forever()
if __name__ == '__main__':
    main()
```

Appendix C – Use of Generative AI

Portions of the software code were developed with assistance from Claude (Anthropic) free version and ChatGPT (Open AI), a generative artificial intelligence language models. The tools were used for code generation, debugging assistance. All generated code was reviewed, tested, and modified by team members to meet project requirements.